プログラミングセミナー

第 5 回　乱数生成

乱数とは

- 出現する値に規則性のない数のこと
- 確率的な要素を持つ現象のシミュレーションには欠かせない



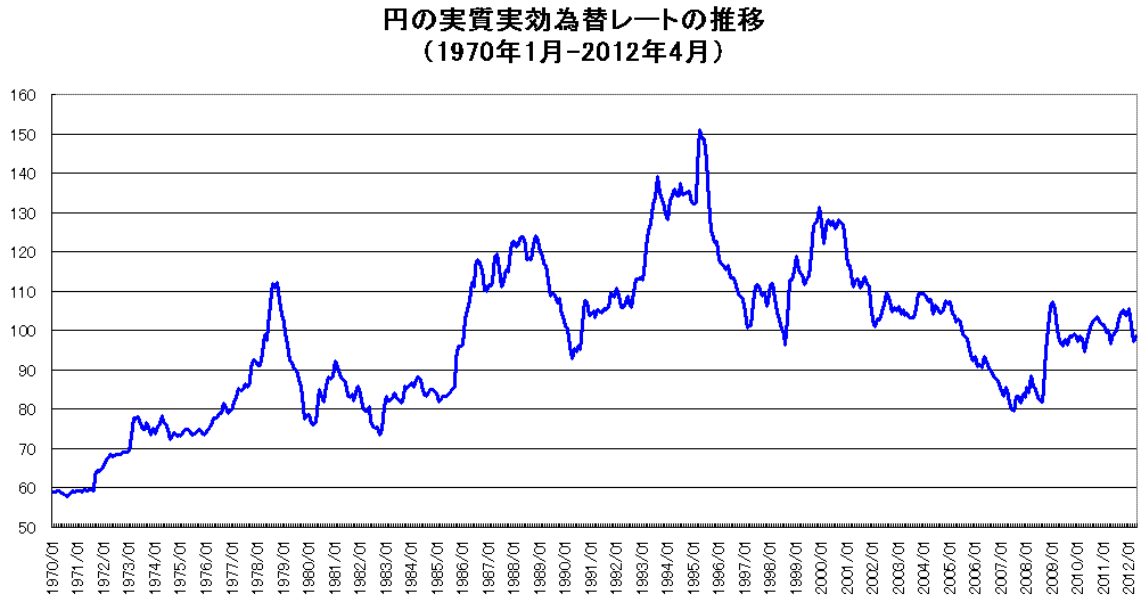
https://onl.la/4y9uShq

乱数の利用例：確率モデル

- 自然・社会現象を理解するうえで確率モデルが良く使われる
  - 待ち行列理論
  - 為替レート



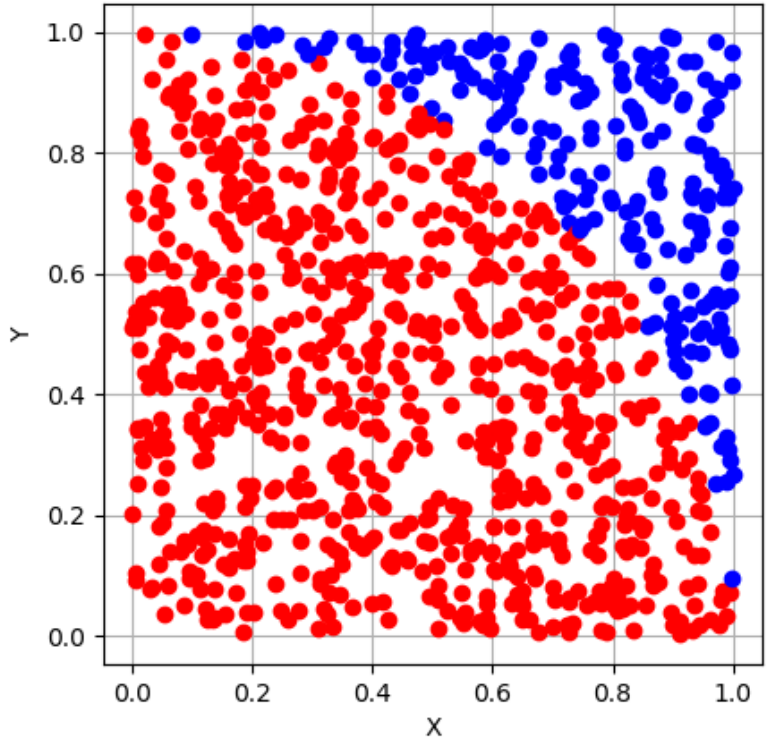
https://onl.la/QdF41Fs



http://stockbondcurrency.blog.fc2.com/blog-entry-3.html

乱数の利用例：モンテカルロ法

- **例：円周率**
  - 1. 1×1の範囲にランダムに点を打つ
  - 2. 円の内部に入っている回数をN_{in}
  - 3. 円周率~ $\frac{N_{in}}{N_{total}}$
- **乱数を使って積分計算ができる**
- **モンテカルロ法は遅いけど手軽**



https://onl.la/3EmCceS

疑似乱数

- ソフトウェアで生成されたランダムは真にランダムではない
- 決定的で予測可能なアルゴリズムに依存している
  - 逆に科学技術計算では重宝される(再現性がある)
- 真正乱数生成器 : 同位体の放射性崩壊や電磁波静的など

https://onl.la/i4qQbbD

一様乱数

- **確率密度関数** $P(x) = \begin{cases} \frac{1}{b-a} & , \quad a \leq x < b \\ 0 & , \quad otherwise \end{cases}$
- **区間** $[0,1)$ **で定義された一様分布乱数がすべてのもとになる**
- **生成法**
  - 線形合同法(※使わないように)
  - **メルセンヌ・ツイスター(※これを使う)**
  - Xorshift

線形合同法 (※使わないように)

- $X_n = aX_{n-1} + c \ \ (mod\ m)$
  - 整数上の一様乱数
  - 剰余(割り算の余り，mod)
  - 単純で高速，昔は使っていた
  - 周期性・系列相関があってもう使うべきではないとされている(高次元空間での結晶化)
  - C言語のrand()はこれ (※絶対に使わないこと)

メルセンヌ・ツイスター(MT, 推奨)

- 松本眞と西村拓士が作成した，高速な疑似乱数生成法
- 世界的にこれが標準な乱数
- 整数値・連続値の両方に対応

メルセンヌ・ツイスター(MT, 推奨)

長所

- 周期が非常に長い ($2^{19937}-1$, メルセンヌ素数)
- 高次元空間 (623次元) でも均等分布
- 統計的にまともな乱数の中では，非常に高速な乱数

メルセンヌ・ツイスター(MT, 推奨)

長所

- 周期が非常に長い ($2^{19937}-1$, メルセンヌ素数)
- 高次元空間 (623次元) でも均等分布
- 統計的にまともな乱数の中では，非常に高速な乱数

短所

- 暗号論的疑似乱数生成器ではない (予測可能)
- 必要なメモリ量が大きい

Xorshift (使ってもいい?)

- 整数上の一様乱数
- 数行で実装出来て速く，統計テストもパス
- オリジナルのMTより速い
- 周期は最低でも $2^{32} - 1$
- 参考 : Google Chromeが使用 ()

注意

- **C言語のrand()は絶対に使わないこと**
- **連続一様分布$u \in [0,1)$を作るときに，離散の一様分布$n = 0,1, \dots, N_{max}$を経由するのは厳密には間違い**

https://takeda25.hatenablog.jp/entry/20130818/1376837093

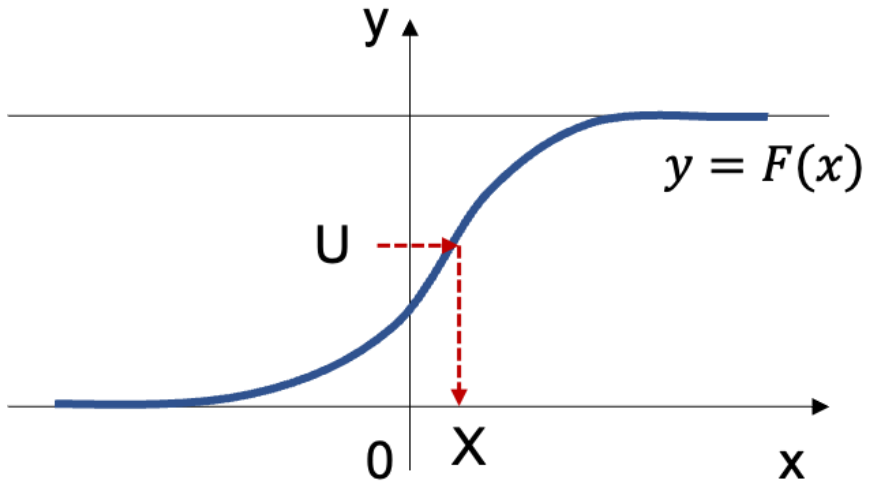
非一様な乱数

生成法

- 一様乱数を基に，様々な手法で生成する
  - 逆関数法 (例 : 指数分布，コーシー分布)
  - 棄却法 (任意の分布)
  - ボックス=ミュラー法 (ガウス分布)
  - エイリアス法 (任意の離散分布)

逆関数法 (※累積分布の解析解が必要)

- **確率密度関数を**$\mathrm{p}(X)$**，累積密度関数を**$\mathrm{P}(X \leq x) \equiv \int_{-\infty}^{x} p(t)dt$ **(以下**$P(x)$**)**
- **区間**$[0,1)$**の一様乱数**u**を使って，**$x = P^{-1}(u)$ **とおくと**x**は**$P(x)$**に従う**
- **指数乱数**
  - $p(x) = \lambda e^{-\lambda x},\ P(x) = 1 - e^{-\lambda x} \Rightarrow\ \ x = \frac{-\log(1-u)}{\lambda}$
- **コーシー乱数**
  - $p(x) = \frac{1}{\pi(1+x^2)},\ P(x) = \frac{1}{\pi}\tan^{-1} x \Rightarrow\ \ x = \tan \pi \left(u - \frac{1}{2}\right)$



https://onl.la/wnR8WpC

逆関数法

区間 $[0,1]$ の一様分布に従う確率変数を Z とする。逆関数によって $Y = h^{-1}(Z)$ が得られるとする。

$$P(Y \leq y) = P(h^{-1}(Z) \leq y)$$

$$= P\big(Z \leq h(y)\big)$$

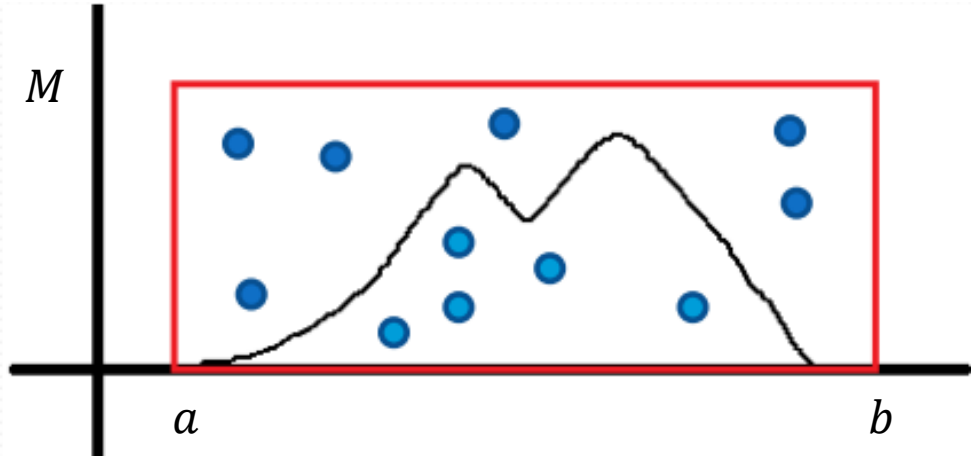
Z が $[0,1]$ 一様分布に従うとき、$P(Z \leq z) = z$ より

$$P(Y \leq y) = h(y)$$

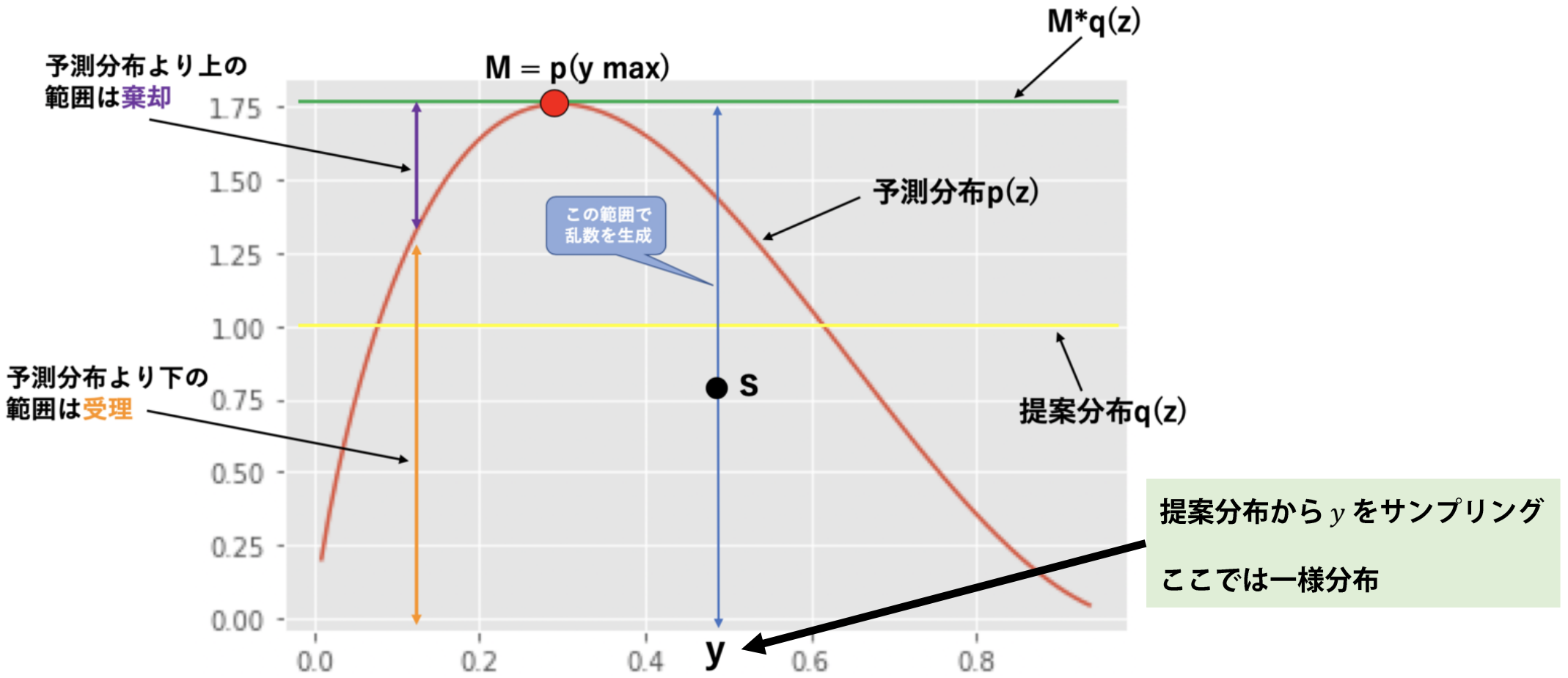
Y は $h(y)$ を累積分布関数に持つ確率分布に従っている。

棄却法

- 確率変数 X の確率密度関数 $f(x)$ が次を満たすとき X に対応する乱数を生成
  - $0 \leq f(x) \leq M \ \ (a \leq x < b),\ f(x) = 0 \ \ (x < a, b \leq x)$
- 任意の確率分布 (特に連続確率分布)に使える (※とても遅い)
- 高速化するには，一様分布ではなく，再現したい分布に似た分布を使う



棄却法

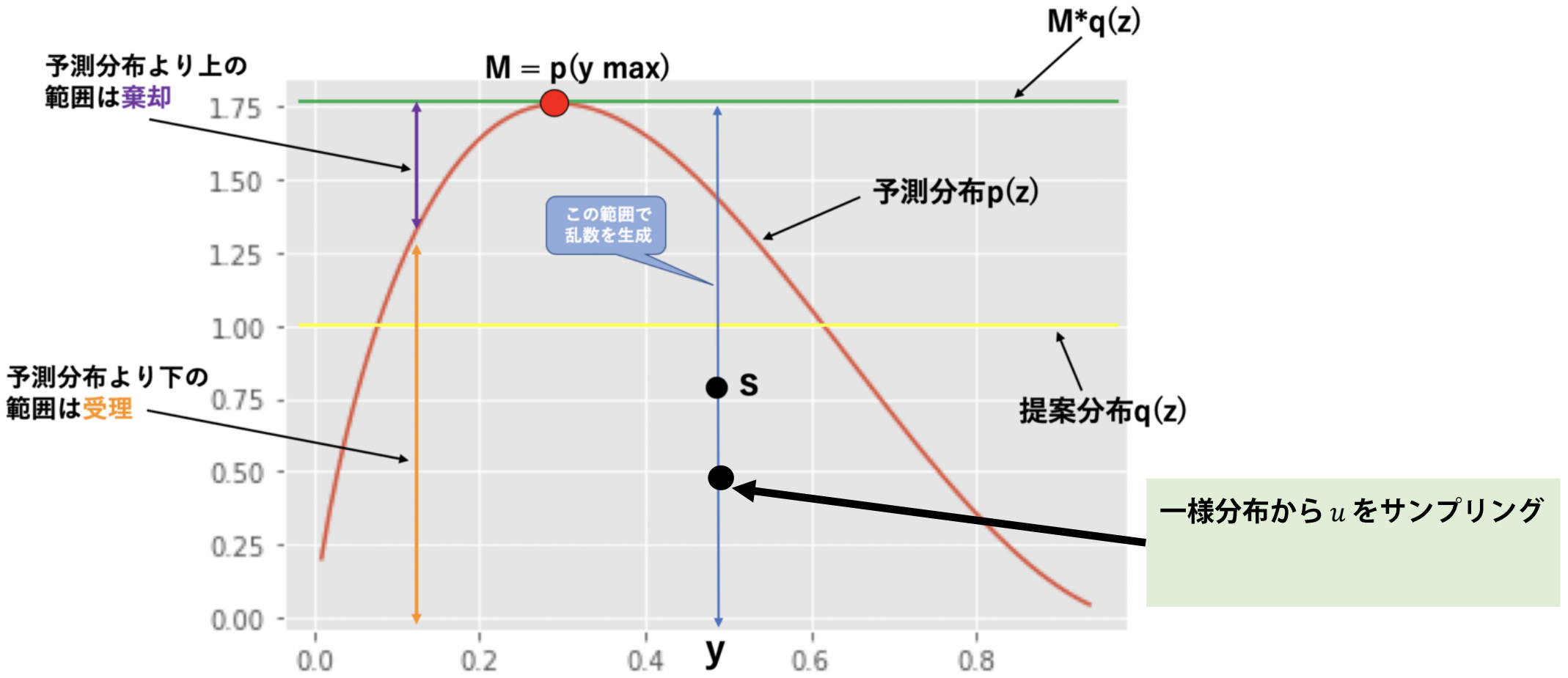


提案分布から y をサンプリング

ここでは一様分布

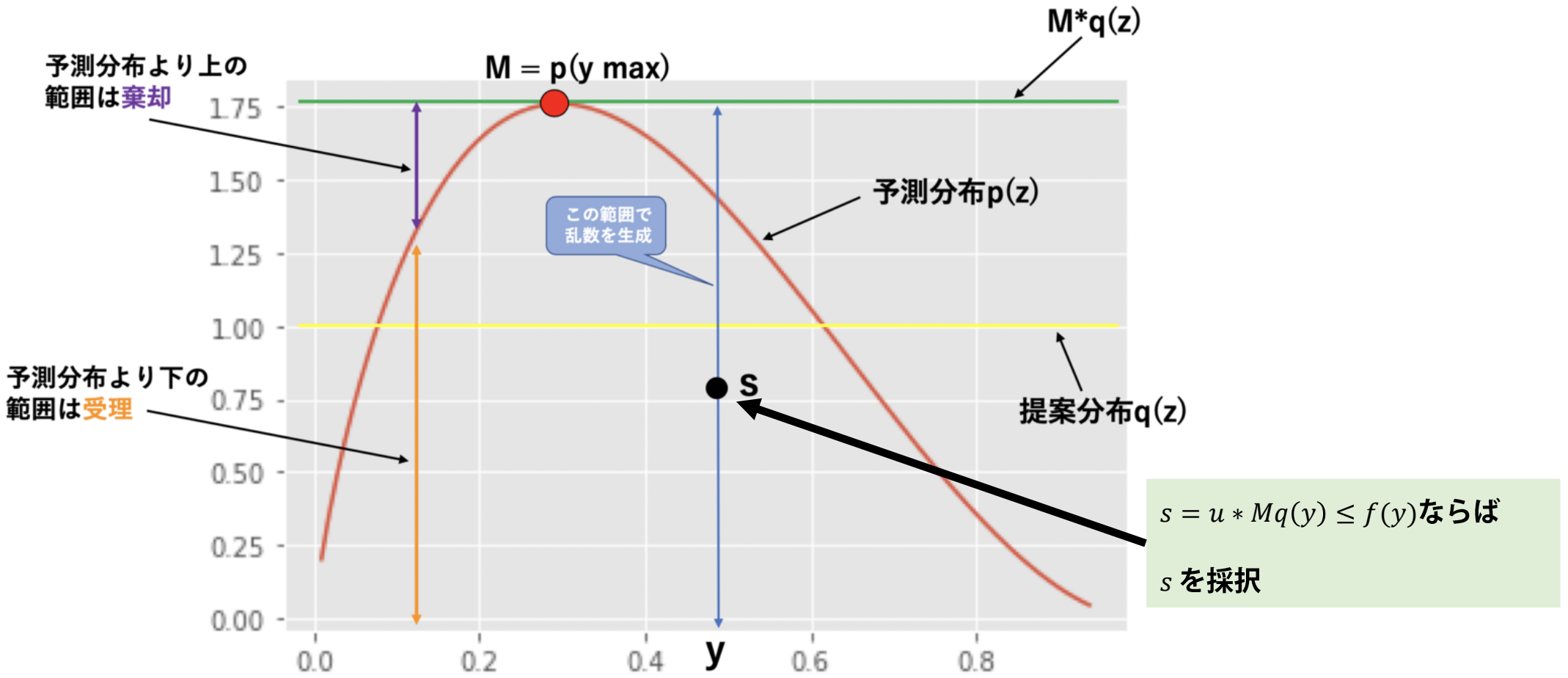
https://onl.la/zuDrmLy

棄却法



https://onl.la/zuDrmLy

棄却法



$s = u * Mq(y) \leq f(y)$**ならば**

s **を採択**

https://onl.la/zuDrmLy

棄却法

確率密度関数が $f(x)$ である確率変数 X を生成したい。$f(x)$ を覆うことができる関数 $\mathrm{M} * g(y)$ を用意する。

$$\forall y, \qquad f(y) \leq M * g(y)$$

確率密度関数が g である確率変数 Y , $(0,1)$ 上の一様乱数に従う確率変数 U を生成する。

$U \leq \frac{f(Y)}{Mg(Y)}$ ならば $X = Y$ として採用。そうでなければ棄却する。

[証明]

$$P\left(Y \leq x | U \leq \frac{f(Y)}{Mg(Y)}\right) = \frac{P\left(Y \leq x, U \leq \frac{f(Y)}{Mg(Y)}\right)}{P\left(U \leq \frac{f(Y)}{Mg(Y)}\right)} = \frac{\int_{-\infty}^{x} \int_{0}^{\frac{f(y)}{Mg(y)}} g(y) du dy}{\int_{-\infty}^{\infty} \int_{0}^{\frac{f(y)}{Mg(y)}} g(y) du dy}$$

$$= \frac{\int_{-\infty}^{x} \frac{f(y)}{Mg(y)} g(y) dy}{\int_{-\infty}^{\infty} \frac{f(y)}{Mg(y)} g(y) dy} = \frac{\int_{-\infty}^{x} f(y) dy}{\int_{-\infty}^{\infty} f(y) dy} = \int_{-\infty}^{x} f(y) dy = P(X \leq x)$$

ボックス=ミュラー法(正規乱数)

$X, Y \in (0,1)$の一様乱数とする．この時，

- $z_1 = \sqrt{-2logX}\cos 2\pi Y$
- $z_2 = \sqrt{-2logX}\sin 2\pi Y$

このはz_1, z_2 平均0，分散1で互いに独立なガウス分布に従う

$$\mathrm{f}(x) = \frac{1}{\sqrt{2\pi\sigma^2}}\exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

最も簡単なのは，STLの標準ガウスノイズを使うこと

(Pythonでもnumpy等で用意されている)

エイリアス法(STLを使うと簡単?)

- 任意の離散分布に使える
- 取りうる値：$x_1, x_2, \ldots, x_n$，とる確率：$p_1, p_2, \ldots, p_n$
- 連続分布には，刻み幅を導入して離散分布に直して使う
- 初期化にO(n)，サンプリングにO(1)の時間がかかる

- 9この容器にたかだか2個の箱が入っている.
- 0から9の整数乱数をサンプリングし容器を選ぶ.
- また乱数を振ってどちらの箱を選ぶか決める.



https://qiita.com/kaityo256/items/1656597198cbfeb7328c

C++ (STL) での実装

C++上で標準ライブラリの乱数を使う

- C言語系列の深刻なデメリット，標準ライブラリがない．．．
- Pythonとかはライブラリが充実している
- C++では標準ライブラリがついている
  - Pythonよりはるかに高速で便利！(らしい!)
- #Standard template library (STL)

整数上の一様乱数 (MT)

- 整数上のMTの生成方法
- クラス std::mt19937 を使う
  - Seed値を固定する
  - 疑似乱数生成器クラス mt(seed)を定義
  - mt() と関数呼び出しで乱数を生成
- 毎回初期化する mt_constの結果も確認
  - mt_const() はすべて同じ値が出力される

```cpp
#include <stdio.h>
#include <random>
#include <iostream>
#include <fstream>


int main()
{
    int seed = 1;
    std::mt19937 mt(seed); // クラス std::mt19937


    for (int i = 0; i < 100; i++)
    {
        std::mt19937 mt_const; // クラス std::mt19937
        std::cout << double(mt()) << ',' << double(mt_const()) << std::endl;
    }

    return 0;
}
```

https://cpprefjp.github.io/reference/random/mt19937.html

実数値の一様乱数

- **実数一様乱数 (MT) の生成**
- **整数MTをベースに一様乱数を生成する**
  - Seed値を固定する
  - 疑似乱数生成器クラス mt(seed) を定義
  - 一様分布生成器クラス uni(min, max) を定義
  - std::uniform_real_distribution<double>にmtを渡して (参照渡し) 一様乱数を生成する
    - uni 自体は分布を表しているだけで、uni 単体で乱数を生成する機構はない
  - mt() を渡そうとすると、mt() で生成した double型の乱数を渡すためエラーになる

```cpp
#include <stdio.h>
#include <random>
#include <iostream>

int main()
{
    int seed = 1;
    std::mt19937 mt(seed); // クラス std::mt19937
    std::uniform_real_distribution<double> uni(0.0, 1.0);

    for (int i = 0; i < 10; i++)
    {
        std::cout << double(uni(mt)) << std::endl;
    }
    return 0;
}
```

https://cpprefjp.github.io/reference/random/uniform_real_distribution.html

ガウス乱数

- ガウス乱数 (MT + ボックス=ミュラー法) の生成
- 整数MTをベースに生成する
  - std::uniform_real_distribution と同様に生成する
  - gauss(平均, 標準偏差)

```cpp
#include <stdio.h>
#include <random>
#include <iostream>

int main()
{
    int seed = 1;
    std::mt19937 mt(seed); // クラス std::mt19937
    std::normal_distribution<double> gauss(0.0, 1.0);

    for (int i = 0; i < 10; i++)
    {
        std::cout << double(gauss(mt)) << std::endl;
    }
    return 0;
}
```

任意の離散分布の生成

- 離散分布 (上の例だと$p_0 : p_1 : p_2 = 1 : 2 : 4$)
- 整数$0,1,2$ が確率 $p_0 : p_1 : p_2$ で生成される
- 連続分布は刻み幅を入れる

```cpp
#include <stdio.h>
#include <random>
#include <iostream>

int main()
{
    int seed = 1;
    std::mt19937 mt(seed); // クラス std::mt19937
    std::discrete_distribution<int> discrete = {1, 2, 4};

    for (int i = 0; i < 10; i++)
    {
        std::cout << double(discrete(mt)) << std::endl;
    }
    return 0;
}
```

課題１：一様乱数

1. [0,1)の一様乱数をN個生成する関数を定義 (C++)
2. 一様乱数を 10^6 個発生し (C++)，そのCDF・PDFを書き出す
3. 一様乱数から中心極限定理を経由して，標準正規分布乱数を作り，PDFをチェック

- $Z \sim N(0,1), U \sim U[0,1)$

$$z_1 = \frac{u_1 + u_2 + \cdots + u_n - n\mu}{\sqrt{n}\sigma}$$

課題1: ヒント

※必要に応じて拡張してください

```
vector<double> generate_uni_vec(int N)
// size Nのvectorを返す関数
// 一度に必要な数の乱数を全て生成することを想定している
{

    // 乱数クラスを定義

    vector<double> outputs;


    for (int i = 0; i < N; i++)
    {
        // 乱数を追加
    }
    return outputs;
}
```

補足：中心極限定理

- 期待値 μ と分散 σ^2 をもつ独立同分布 (i.i.d.) に従う確率変数列 $X_1, \ldots$ に対し $S_n = \sum_{i=1}^{n} X_n$ とおくと，
  - $P\left(\frac{S_n - n\mu}{\sqrt{n}\sigma} \leq \alpha\right) \rightarrow \frac{1}{\sqrt{2\pi}} \int_{\infty}^{\alpha} e^{-\frac{x^2}{2}} dx \qquad (n \rightarrow \infty)$
- 独立同分布に従う確率変数列の部分和を標準化すると，期待値 0 , 分散 1 の正規分布 $N(0,1)$ に分布収束する
- Nが十分大きいと部分和 S_n は平均 $n\mu$，分散 $n\sigma^2$ の正規分布 $N(n\mu, n\sigma^2)$ に収束し，標本平均は $N\left(\mu, \frac{\sigma^2}{n}\right)$ に従う

課題 2 : ガウス乱数

1. 平均 μ , 標準偏差 σ のガウス乱数をN個生成する関数を定義 (C++)
2. 正規乱数を 10^6 個ファイル出力して (C++), そのCDF・PDFを書き出す
3. 正規分布の和がガウス分布になることを確認
   - $X \sim N(\mu_1, \sigma_1^2), Y \sim N(\mu_2, \sigma_2^2)$ (互いに独立), $X + Y \sim N(\mu_1 + \mu_2, \sigma_1^2 + \sigma_2^2)$
   - 暇だったら積はどうなるのか確認
4. ランダムウォーク時系列を作ってみよう
   - $X_n\ (n = 1,2, \dots) \sim_{i.i.d.} N(0,1),\ S_n = X_1 + \cdots + X_n$

課題 3 : 指数乱数

- $\lambda = 1$ の指数乱数を様々な方法で生成してCDF・PDFをプロット

1. 逆関数法を用いて生成
2. 棄却法を用いて生成
3. 離散分布に直して生成